LAB W14 · ONE LAB, NINETEEN STEPS, EACH ONE BUILDS ON THE LAST

Paws & Whiskers: who are you, and is this yours?

A pet shelter's website. Anyone can see the pets waiting for a home. People who adopted one should be able to log in and see their own pet's private record. Tonight you add logins with Better Auth, straight from its own documentation, on a brand-new database. Then you make the API ask two questions before it hands over anything private: **can you get in?** and **is this yours?**

This lab is meant to stretch you. Every step tells you what to build and how to check it. If you get stuck, every step links to the full code for that step in the Code reference at the back. Try it yourself first, then look.

Better Auth is a library: code someone else wrote, that you install and use. On Tuesday we wrote logins by hand: hashing passwords, a sessions table, a cookie, expiry. Better Auth does all of that for us, more carefully. Part of tonight is learning to read its documentation, so several steps send you to the Better Auth website and tell you which part to read. Keep that tab open.

Contents

Part 1 · A new database

1. A new Neon project, and `.env.local`
2. The shelter's tables: `db/init.sql`
3. Run it, and find the open door

Part 2 · Better Auth: who are you?

4. Install Better Auth
5. The secret and the address
6. `lib/auth.ts`
7. Better Auth's tables
8. The route that answers `/api/auth/...`
9. Close the doors you don't use
10. The browser half: `lib/auth-client.ts`
11. The login screen

Part 3 · The API: can you get in? Is this yours?

12. Can you get in? `/api/my-pets`
13. Is this yours? `/api/my-pets/4`
14. The five cases

Part 4 · Together: protect the page

15. The cookie way: `proxy.ts`
16. Break it with a fake cookie
17. The `UserContext` wrapper
18. The layout fills it in
19. The pages use it, and the proxy goes

What "done" looks like · Going further · Things that will bite you · Code reference

Words you'll see tonight

WORD	WHAT IT MEANS HERE
Authentication	"Who are you?" Proving you're the person you say you are, usually with a password. Better Auth's job.
Authorization	"What are you allowed to do?" Deciding whether this person may see or change this thing. Always our job.
Session	A row in the database that says "this browser is logged in as this person, until this time." Tuesday's coat-check ticket.
Cookie	A small piece of text the server asks the browser to keep and send back with every request. Here it holds the session's ticket number.
401	"I don't know who you are." Not logged in.
404	"There's nothing here." We also send it for somebody else's pet, so we never admit it exists.
Context (React)	A value that every component inside a wrapper can read, without passing it down by hand. Part 4.

Who's who in the database

`db/init.sql` already says who owns which pet. You'll sign these people up yourself in Step 11. The passwords are made up for this lab; use them so your classmates' answers match yours.

NAME	EMAIL	PASSWORD	OWNS
Ana Ortiz	`ana@example.com`	`ana-password-1`	Duke (4), Clementine (5)
Ben Carter	`ben@example.com`	`ben-password-1`	Juniper (7), Noodle (8)
Dr. Sam Lee	`dr.lee@example.com`	`vet-password-1`	Nobody. A vet. Only for Going further.

Pets 1, 2, 3 and 6 are still at the shelter. Nobody owns them.

1 A new Neon project, and `.env.local`

This lab gets its **own** Neon project, not the Corner Store one. A project is a separate database server with its own address and password. Every real app you start begins with an empty database, so tonight you see the whole setup from zero, and nothing you do can break your store.

1. Go to console.neon.tech and log in, the same way as in week 12.
2. You land on your list of projects. Click **New project**. (If you land inside your store project instead, click the project name at the top left, then **New project**.)
3. Fill in the form:
   - **Project name:** `pet-shelter`
   - **Postgres version:** leave it as it is.
   - **Cloud provider:** AWS.
   - **Region: AWS US East 1 (N. Virginia)**, the closest one to us. Any US East region is fine.
   - If there's an option for **Neon Auth**, leave it **off**. Neon Auth is Neon's own login feature. We're adding Better Auth ourselves, so you can see every piece.
4. Click **Create**. You land on the new project's dashboard.
5. Click **Connect** (top right of the dashboard). A box opens. Leave every dropdown as it is: database `neondb` is the right one this time, because this project only has one.
6. Click **Show password**, then copy the connection string. It starts with `postgresql://`.

> **NEON SAYS YOU'VE REACHED YOUR PROJECT LIMIT?**
>
> Neon's free plan only allows a few projects. Delete one you don't need any more (that project's **Settings** → **Delete project**), but **not** your Corner Store one. Not sure? Ask before you delete anything.

Now put that string where the app can read it:

1. In VS Code, make a new file called `.env.local` in the `week14/lab1` folder, next to `package.json`. Make it **in VS Code**, not Notepad, so it doesn't secretly become `.env.local.txt`.
2. Copy the three lines from `.env.example` into it.
3. Paste your string after `NEON_DATABASE_URL=`. No quotes, no spaces.
4. In the string, find `sslmode=require` and change it to `sslmode=verify-full`.

```
NEON_DATABASE_URL=postgresql://neondb_owner:****@ep-something-pooler.us-east-1.aws.neon.tech/neondb?
sslmode=verify-full&channel_binding=require
BETTER_AUTH_SECRET=
BETTER_AUTH_URL=
```

Why change `sslmode`? `sslmode` says how carefully the app checks it's really talking to Neon before sending the password. This lab talks to Neon with a package called `pg`, and when `pg` sees `require` it prints a long `SECURITY WARNING` every time the app starts. It's warning that the meaning of `require` will get weaker in a future version. `verify-full` is the strict check it already does today, written out, so the warning goes away and nothing else changes. Leave the two lines below it empty until Step 5.

CHECK IT

Run `git status`. `.env.local` must **not** be listed. If it is, stop and ask: that string is a password, and this repo is public.

2 The shelter's tables: `db/init.sql`

The new database is empty. `db/init.sql` makes the shelter's two tables, `pets` and `vets`, and fills them. Open it in VS Code and read the comments first: they say which columns are public and which are private.

1. In Neon, on your `pet-shelter` project, click **SQL Editor** on the left.
2. Copy **all** of `db/init.sql`, paste it in, and click **Run**.

CHECK IT

The result at the bottom shows **8 pets**: 4 marked `(at the shelter)`, 2 for `ana@example.com`, 2 for `ben@example.com`.

Now click **Run** again. Same 8 rows, no error, no duplicates. The file only makes what's missing, so running it twice is safe.

Notice what's missing: there's no table of users, no passwords, no sessions. Those are Better Auth's tables, and Better Auth will write them for you in Step 7. Look at `owner_email` in the file: a pet's owner is written as an email, because nobody has signed up yet, so there's no user to point at. The comment in the file says more.

3 Run it, and find the open door

```
cd week14/lab1
npm install
npm run dev
```

Open http://localhost:3000. Click around:

PAGE	WHAT YOU SHOULD SEE
`/`	Biscuit, Miso, Pepper and Waffles. The public page. It works.
`/my-pets`	`501: Not built yet. That's Step 12.` The page opens for anyone.
`/login`	A form. Every button says "Not built yet."

Now, before you click it, guess: **you are not logged in. Nobody can log in yet. What happens when you open localhost:3000/api/my-pets/4?**

Open it. That's Duke's private record: Ana's email, his microchip number, his vet notes. Change the `4` to `7`: Ben's cat. Anyone on the internet could walk through every pet just by counting. **This is the open door you'll close in Step 13.** Open `app/api/my-pets/[id]/route.ts`: nothing in it asks who's calling.

4 Install Better Auth

Open the Better Auth docs in a new tab: **better-auth.com/docs/installation**. This page is the recipe for Steps 4 to 10. Read the **Install the Package** part, then stop your app (`Ctrl + C`) and run:

```
npm install better-auth
```

The page also shows `pnpm`, `yarn` and `bun`. Those are other tools that do the same job as `npm`. We use `npm`.

CHECK IT

Open `package.json`. Under `"dependencies"` there's a new line, `"better-auth"`, with a version number. That line is how the next person to run `npm install` gets it too.

5 The secret and the address

In the docs, read **Set Environment Variables**. Better Auth needs two values in `.env.local`:

- `BETTER_AUTH_SECRET`: a long random password that only your app knows. Better Auth uses it to **sign** each login cookie: it mixes the cookie's value with the secret and attaches the result, so if anyone changes the cookie, the signature stops matching and Better Auth refuses it.
- `BETTER_AUTH_URL`: the address your app runs at. Better Auth uses it to refuse requests sent from other websites.

The docs page has a **Generate Secret** button, and suggests an `openssl` command, which Windows doesn't have. This works on every computer:

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

Paste what it prints after `BETTER_AUTH_SECRET=`, and fill in the address:

```
BETTER_AUTH_SECRET=6f1c...a long line of letters and numbers...9e2b
BETTER_AUTH_URL=http://localhost:3000
```

Your secret is yours: don't paste it in Slack, in a screenshot, or into a prompt. A deployed app gets its own, different secret.

6 `lib/auth.ts`

In the docs, read **Create A Better Auth Instance**, then **Configure Database** (click the **postgres** tab), then **Authentication Methods**. Together they say: make a file called `auth.ts`, give Better Auth a database, and turn on email and password.

Make a new folder `lib` next to `app`, and in it a file `auth.ts`. The docs make a brand-new `Pool` (a set of open connections to the database). We already have one in `providers/neon/neonDatabaseProvider.ts`, so Better Auth gets ours:

lib/auth.ts

```
import { betterAuth } from "better-auth";
import { NeonDatabaseProvider } from "../providers/neon/neonDatabaseProvider";

export const auth = betterAuth({
  database: NeonDatabaseProvider.getInstance().getPool(),
  emailAndPassword: { enabled: true },
});
```

That's the whole thing. Everything from Tuesday (hashing, sessions, the cookie, expiry, logging out) now lives behind those few lines.

Stuck? C1 has the file with comments.

7 Better Auth's tables

Better Auth needs four tables of its own. In the docs, read **Create Database Tables**, then follow its link to the **Core Schema** and read what each table is for:

TABLE	ONE ROW PER	TUESDAY'S VERSION
`user`	person	the `users` table
`session`	logged-in browser	the `sessions` table
`account`	way that person logs in. Email and password is one; "Sign in with Google" would be another. **The hashed password lives here.**	the `password_hash` column
`verification`	short-lived code, like a "check your email" link	none

You don't type these tables out. The docs give two commands: `migrate` makes the tables straight in your database; `generate` writes the SQL to a file so you can read it first and run it yourself. **We use** `generate`: you see exactly what's about to change in your database before it happens, which is how it's done on a real team. Run it in the `lab1` folder:

```
npx auth@latest generate
```

If it asks to install the `auth` package, or whether to write the file, type `y` and press Enter. It reads `lib/auth.ts`, connects to your database using `.env.local`, sees what's missing, and writes a file. **Expect some scary-looking output that's fine:**

```
Warning: Module type of file:///.../lib/auth.ts is not specified ...
ERROR [Better Auth]: Database schema mismatch
  Missing tables
    user, session, account, verification
  help: Run `npx auth migrate` to add the missing tables and columns.
🚀 Schema was generated successfully!
```

The red `ERROR` is Better Auth looking at your database and listing the four tables it doesn't have yet. That's exactly what the file it just wrote will fix. The last line is the one that matters.

There's a new folder, `better-auth_migrations`, with one `.sql` file in it, named with today's date and time. **Open it and read it.** It's four `create table` lines and three `create index` lines (an index is what lets the database find one row fast without reading them all). See C2 for what yours should look like.

1. Copy the whole file.

2. Neon → **SQL Editor** on your `pet-shelter` project → paste → **Run**.

CHECK IT

In Neon, click **Tables** on the left. You should see six: `pets`, `vets`, and Better Auth's `user`, `session`, `account`, `verification`. All four of Better Auth's are empty.

Running the file a second time says `relation "user" already exists`. That just means you already did it.

ASK YOUR CLAUDE

> *"Here's a SQL file a library generated for my Postgres database: [paste it]. Explain each table in plain words. Why are some column names in double quotes, like "userId"? What does 'references "user" ("id") on delete cascade' do?"*

8 The route that answers `/api/auth/...`

In the docs, read **Mount Handler** and click the **Next.js** tab (the App Router one, not Pages Router). Better Auth answers a lot of addresses: `/api/auth/sign-in/email`, `/api/auth/sign-up/email`, `/api/auth/sign-out`, and more. You don't write a route for each. One file with a special name catches them all.

Make the folders `app/api/auth/[...all]/` (yes, with the square brackets and the three dots) and in it `route.ts`, with the code from the docs:

app/api/auth/[... all]/route.ts

```
import { auth } from "@/lib/auth";
import { toNextJsHandler } from "better-auth/next-js";

export const { POST, GET } = toNextJsHandler(auth);
```

`[...all]` means "anything under here, however many slashes." `@/lib/auth` is a shortcut: `@/` means "the top folder of this project," so you don't have to count `../`. It's set up in `tsconfig.json`.

Start the app again (`npm run dev`).

CHECK IT

Open localhost:3000/api/auth/get-session. It says `null`: Better Auth is answering, and nobody is logged in.

Now open localhost:3000/api/auth/ok. It says `{"ok":true}`. Did you ask for that address? Remember it for the next step.

9 Close the doors you don't use

That one line from the docs opens **every** address Better Auth knows. That's about thirty of them, including `/update-user`, `/delete-user`, `/change-password` and `/list-sessions`. Our pages will call four. Every open address is one more thing somebody can poke at, and you'd never know, because you never wrote it.

Tuesday's rule was **deny by default**: start with everything closed and open only what you need. Do it here:

1. Replace `route.ts` with the version in C4. It keeps a short list of the four addresses we use, and answers `404` to everything else without ever asking Better Auth.
2. In `lib/auth.ts`, add the `disabledPaths` list from C1. That tells Better Auth itself to switch those addresses off too. It's a second lock behind the first: if somebody later loosens the list in `route.ts`, the riskiest ones stay shut.

CHECK IT

OPEN	BEFORE	NOW
`/api/auth/ok`	`{"ok":true}`	**Not found**
`/api/auth/list-sessions`	`Unauthorized` (it's there, you just weren't let in)	**Not found**
`/api/auth/get-session`	`null`	`null` **(still open: we use it)**

When you add a feature that needs another Better Auth address, add that one address to the list. If something you expect to work answers `Not found`, this list is the first place to look.

10 The browser half: `lib/auth-client.ts`

In the docs, read **Create Client Instance** and click the **React** tab. `lib/auth.ts` runs on the server. The pages run in the browser and need their own half, which sends requests to `/api/auth/...` for them.

lib/auth-client.ts

```
import { createAuthClient } from "better-auth/react";

export const authClient = createAuthClient();
```

The docs pass `baseURL: "http://localhost:3000"`. Their own comment says it's optional when the pages and the API are the same website, which ours are. Leaving it out means it still works once the app is deployed somewhere that isn't `localhost`.

11 The login screen

Read **Basic Usage → Email & Password** in the docs. It shows three calls: `authClient.signUp.email`, `authClient.signIn.email`, and `authClient.signOut`.

Open `app/login/page.tsx`. The form is built. Fill in the three functions:

1. At the top, import `authClient` from `"../../lib/auth-client"`.
2. `signUp`: call `authClient.signUp.email({ name, email, password })`. You get back `{ data, error }`. If there's an `error`, show `error.message`. If not, show `"Signed up and logged in as "` plus `data.user.name`.
3. `logIn`: the same with `authClient.signIn.email({ email, password })`.
4. `logOut`: `await authClient.signOut()`, then show `"Logged out"`.

Stuck? C6 is the whole page.

Now sign people up, using the table at the top of this handout:

1. On `/login`: name `Ana Ortiz`, email `ana@example.com`, password `ana-password-1`, click **Sign up**.
2. **Log out.** Sign up `Ben Carter` the same way, then log out again.
3. Log in as Ana: email and password only, click **Log in**.

CHECK IT: LOOK IN THREE PLACES

- **DevTools** (`F12`) → **Application** → **Cookies** → `http://localhost:3000` : a cookie called `better-auth.session_token`. Its **HttpOnly** box is ticked, which means JavaScript in the page can't read it. Only the browser sends it.
- **Neon SQL Editor:** `SELECT * FROM "user";` shows Ana and Ben, each with a long random `id` that Better Auth made up. (Don't forget the double quotes. See Things that will bite you.)
- **Neon SQL Editor:** `SELECT "userId", "providerId", left(password, 25) FROM account;` shows one row each, `credential`, and a password that's a jumble. That's the hash. Not even you can read it back.
- Open /api/auth/get-session: the session and the user, with an `expiresAt` seven days away.

Now try to get it wrong, and read each message:

TRY	MESSAGE
Ana with the password `wrong-password`	`Invalid email or password`
`nobody@example.com` with any password	`Invalid email or password`: the **same** message. Wednesday's lesson: it never tells an attacker which emails have accounts.
Sign up with the password `short`	`Password too short` (8 characters at least)

The terminal running `npm run dev` says what really happened (`Invalid password`, `User not found`). The page doesn't. That's on purpose.

12 Can you get in? `/api/my-pets`

Logins work. But the API still doesn't ask who's calling. First, one file whose job is to answer "who is this?", the same job as Tuesday's `authService`. Make `services/authService.ts` (full file in C7). The part that matters:

```
const result = await auth.api.getSession({ headers: await headers() });
```

`headers()` (from `"next/headers"`) is the request that just came in, cookie included. `auth.api.getSession` reads the cookie, checks its signature with your secret, looks the session up in the `session` table, and checks it hasn't expired. All four pass: you get the user back. Any one fails: `null`. Your method gives back `{ id, name, email }`, or `undefined` when nobody is logged in.

Then open `app/api/my-pets/route.ts` and replace the TODOs:

1. Ask `AuthService.getInstance().getCurrentUser()` who this is.
2. Nobody? Answer `{ error: "Log in first" }` with status `401`.
3. Somebody? Answer with `PetService.getInstance().getPetsOwnedBy(user.email)`.

Where does "my" come from? From the session. Never from the request: there's no `?email=` on the address for anyone to change to somebody else's.

CHECK IT

Logged in as Ana, open `/my-pets`: Duke and Clementine, with their microchips and vet notes. Log out and open it again: `401: Log in first`. Log in as Ben: Juniper and Noodle.

Stuck? C7 and C8.

13 Is this yours? `/api/my-pets/4`

Now the open door from Step 3. Open `app/api/my-pets/[id]/route.ts`. It needs both questions, **in this order**:

1. **Can you get in?** The same three lines as Step 12: no user, `401`.
2. **Is this yours?** After the pet is looked up: if there's no pet, **or** `pet.ownerEmail` isn't `user.email`, answer `404` with `No pet of yours with id ...`.

Why 404 and not 403? `403` means "it's here, and you can't have it." That tells somebody counting through ids which ones exist. `404` for both "doesn't exist" and "not yours" gives nothing away. Same choice as Tuesday's receipts.

Before you test, guess: **Ana is properly logged in. She isn't faking anything. What happens when she changes 4 to 7?**

CHECK IT: LOGGED IN AS ANA

OPEN	WHOSE	ANSWER
`/api/my-pets/4`	Ana's (Duke)	**200, the record**
`/api/my-pets/7`	Ben's	**404**
`/api/my-pets/1`	Nobody's, still at the shelter	**404**
`/api/my-pets/99`	Doesn't exist	**404**

Log out, and `/api/my-pets/4` is **401**.

Notice what made the difference: Ana was logged in for all four. **Authentication did its whole job every time. That's the other question, and it's ours.**

Stuck? C9.

14 The five cases

A login that "works" has only been tested once. Here are five ways to test it on purpose, including two attacks. Run each one and tick it. Then be ready to show **one** of them failing and passing, on your own screen, in your own words.

#	THE CASE	HOW TO RUN IT	EXPECT	✓
1	**The right person gets in**	Log in as Ana, open `/my-pets`	Duke and Clementine	☐

2	**Nobody logged in is turned away**	Log out, open `/api/my-pets/4`	`401`	☐
3	**Logged in, but not yours, is turned away**	As Ana, open `/api/my-pets/7`	`404`	☐
4	**A fake or changed session is turned away**	As Ana, DevTools → Application → Cookies → double-click the `better-auth.session_token` value and add an `x` to the end. Open `/api/my-pets`	`401`. The signature no longer matches	☐
5	**A malicious input is handled**	(a) Open `/api/my-pets/4 OR 1=1` (b) On `/login`, email `' OR '1'='1`, any password, **Log in**	(a) `404`, not every pet (b) `Invalid email`	☐

Why 5(a) is safe: `"4 OR 1=1"` isn't a whole number, so `getPetRecord` never even asks the database. Even if it did, the id goes in as `$1`, a value and never part of the SQL. That's Wednesday's Bobby Tables, closed twice.

A sixth, if you have time: log in as Ana, and in DevTools copy the cookie's value somewhere. Log out. Put the old value back in the cookie and open `/api/my-pets`. `401`: logging out deleted the session row, so the old ticket is worth nothing. In Neon, `SELECT count(*) FROM session;` before and after shows it. That's Tuesday's "session that never expires," fixed.

15 Together: the cookie way, `proxy.ts`

Part 4 is done together, on the projector. Follow along, or watch and come back to it at home.

The API is locked. But the **page** `/my-pets` still opens for anyone, then shows an error. It would be nicer to send logged-out people to `/login` before the page even loads. Week 11's `proxy.ts` runs before every request, so it's the obvious place.

The docs show how: **Integrations → Next.js**, the **Middleware** part. (`proxy.ts` is what Next 16 calls what older versions called `middleware.ts`.) Better Auth gives you `getSessionCookie`, which looks for the login cookie. Make `proxy.ts` at the top of the project, next to `package.json`, from C10.

CHECK IT

Log out. Click **My pets**. You land on `/login` instead. Log in as Ana, click **My pets**: her pets.

16 Break it with a fake cookie

Read the docs again, just below that code. There's a warning: `getSessionCookie` **"only checks for the existence of a session cookie; it does not validate it."** They mark that approach **NOT SECURE**.

Guess first: **the proxy says "no cookie, no entry." What if I just make one up?**

1. Log out.
2. DevTools → **Console**. Type this and press Enter:

   ```
   document.cookie = "better-auth.session_token=nope; path=/"
   ```

3. Click **My pets**.

The page opens. The proxy saw a cookie with the right **name** and let you through. It never asked the database whether `nope` was a real session. What does the page show? `401: Log in first`. **The page let a stranger in. The data didn't**, because the API asks the real question. That's why the API check from Steps 12 and 13 is the one that matters.

Leave the fake cookie there for Step 19.

17 The `UserContext` wrapper

So the page needs to know who you **really** are, checked against the database the way the API does it. And it isn't only this page: the header should say "Hi, Ana," the login page should say who's logged in. Every one of them asking separately would be a mess.

React context is made for this. You put a value into a **wrapper** high up, around the whole app, and any component inside can read it with `useContext`, without anyone passing it down by hand through every component in between.

Make `app/UserContext.tsx` from C11. It has three parts:

- `User`: what we know about the person. `id`, `name`, `email`.
- `UserContext`, made with `createContext<User | null>(null)`. `null` means nobody is logged in.
- `UserProvider`, the wrapper. It takes a `user` and puts it into the context for everything inside it.

Why a separate wrapper component? The layout is a **server component**: it runs on the server and can ask the database things. Context only works in the browser, in a **client component** (that's what `"use client"` at the top means). So the layout finds out who you are, and hands it to this small client component to share.

18 The layout fills it in

Open `app/layout.tsx` and change it to C12. Three changes:

1. `export default async function RootLayout` : it waits for an answer now.
2. First line inside: `const user = await AuthService.getInstance().getCurrentUser();` . The **same** real check the API uses: cookie, signature, session table, expiry.
3. Wrap everything inside `<body>` in `<UserProvider user={user ?? null}>` . (`??` means "if it's `undefined` , use `null` instead.") The **Log in** link becomes `{user ? "Hi, " + user.name : "Log in"}` .

CHECK IT

Log in as Ana and reload any page: the header says **Hi, Ana Ortiz**.

19 The pages use it, and the proxy goes

`app/my-pets/page.tsx` (C13): get the user with `const user = useContext(UserContext);` . Only call the API when there **is** a user. When there isn't, show "*This page is only for people who adopted a pet. Log in to see yours.*" and a link to `/login` . The heading can say whose pets these are, too.

`app/login/page.tsx` (C14): the top line comes from the context now, "*Logged in as Ana Ortiz (ana@example.com)*" or "*Not logged in*". After a successful log in, sign up or log out, call `router.refresh()` (from `useRouter` in `"next/navigation"`).

Why `router.refresh()` ? The layout found out who you are when the page loaded, before you clicked **Log in**. `refresh()` runs the layout again, so it asks again and the context, the header and this page all catch up. Forget it, and you log in and the page still says "Not logged in."

Now **delete** `proxy.ts` . The page doesn't need it any more, and a check that a fake cookie walks through isn't worth keeping.

Your fake cookie from Step 16 is still there. Guess first, then click **My pets**.

CHECK IT

WHO	HEADER	`/MY-PETS` SHOWS
The fake `nope` cookie	Log in	"This page is only for people who adopted a pet." The API isn't even asked.

Logged in as Ana	Hi, Ana Ortiz	Ana Ortiz's pets
Logged out	Log in	"This page is only for people who adopted a pet."

The cookie was a ticket stub. The context is the person, checked. The proxy asked "is there a ticket?" The layout asks "whose ticket, is it real, has it run out?" and then every page knows the answer.

And the API check stays. The page check is for the person using it: a friendly message instead of an error. The API check is the lock. Somebody can skip your page completely and call `/api/my-pets/4` straight from Postman. Only the API stops them.

Clear the fake cookie when you're done:

```
document.cookie = "better-auth.session_token=; path=/; max-age=0"
```

What "done" looks like

- Your own Neon project, `pet-shelter`, with `pets`, `vets`, and Better Auth's four tables, made from the SQL Better Auth generated.
- Ana and Ben can sign up, log in and log out on `/login`. The home page works for anyone.
- `/api/my-pets` is `401` for nobody and the right pets for each person.
- `/api/my-pets/[id]` is `401` for nobody, `404` for somebody else's pet, and the record for your own.
- All five cases in Step 14 pass, and you can show one failing and passing.
- `/api/auth/ok` is `Not found`.
- `/my-pets` knows who you are from `UserContext`, and a fake cookie doesn't fool it.
- `git status` doesn't list `.env.local`. `npm run build` finishes with no errors.

Going further

1. **Vets can see every pet.** Sign up Dr. Sam Lee (`dr.lee@example.com`, `vet-password-1`). `db/init.sql` already put that email in the `vets` table. Make `/api/my-pets/[id]` answer for a pet that's yours **or** when you're a vet. Better Auth has never heard of vets: this is a row in **our** table, checked by **our** code. Add an `isVet(email)` method to `AuthService`. Ask the cheap question (is it

yours?) first, and only ask the database about vets when it isn't. **Restart** `npm run dev` after adding the method. Answer in C15.

2. **Swap seats and attack.** Sit at a classmate's computer, with their app running. Run all five cases from Step 14 against **their** app, then try anything else you can think of: other ids, other Better Auth addresses, editing the cookie. Attacking somebody else's code is a better test than attacking your own, because you don't know where the checks are.
3. **Rename your pet.** Add a `PATCH` to `/api/my-pets/[id]` that changes a pet's name, and a box on `/my-pets` to use it. It needs the same two checks, plus checking the body (week 11). Who should be allowed: the owner, a vet, or both?
4. **Read your own tables.** In Neon: `SELECT * FROM session;`. What's in `ipAddress` and `userAgent`? Log in from a second browser: how many rows does Ana have now? What would you build with that?

Things that will bite you

- `SELECT * FROM user;` **gives back one row that says** `neondb_owner`**.** `user` is a word Postgres already uses: it means "the database login running this query." Put the table name in double quotes: `SELECT * FROM "user";`. Same for Better Auth's camelCase columns: `"userId"`, `"expiresAt"`.
- **Restart after changing** `.env.local`**.** Next only reads it when it starts.
- **Restart after adding a method to a service.** The dev server keeps the copy of `AuthService` it already made, and the old copy doesn't have your new method: `isVet is not a function`. Stop it with `Ctrl + C` and `npm run dev` again.
- **A** `SECURITY WARNING` **about SSL modes** every time the app starts: you skipped changing `sslmode=require` to `sslmode=verify-full` in Step 1. It still works; the warning is just noise.
- `Not found` **from** `/api/auth/...` after Step 9: the address isn't on your list in `route.ts`. Check for a typo before adding anything.
- `Invalid origin` when you log in: `BETTER_AUTH_URL` doesn't match the address in your browser. If Next said **port 3001** because 3000 was taken, either stop the other app or change `BETTER_AUTH_URL` to `http://localhost:3001` and restart.
- **Postman and** `/api/auth/...`**:** Better Auth refuses a `POST` with no `Origin` header once there's a cookie (`403 Missing or null Origin`). Browsers always send one; Postman doesn't. Add a header `Origin` with the value `http://localhost:3000`. Our own `/api/my-pets` routes don't need it.
- **You logged in, and the page still says "Not logged in"** (Part 4): `router.refresh()` is missing after the log in.

- `npx auth@latest generate` **can't connect**: it reads `NEON_DATABASE_URL` from `.env.local`. Run it from the `lab1` folder, and check the string.
- **Claude writes the tables by hand, uses Prisma or Drizzle, or puts `getSession` in every route.** Remind it: Better Auth on a `pg` Pool, tables from `npx auth@latest generate`, and one `AuthService` that the routes ask.
- **The first `npm run dev` writes `AGENTS.md`.** Notes from Next for AI coding tools. Leave it.

Code reference

The code at the end of each step. Try the step first; come here when you're stuck or want to compare. Each block says which file it goes in and links back to its step.

C1 · Better Auth on our database

↑ back to Step 6 · Step 9 adds `disabledPaths`

`lib/auth.ts`

```
// Better Auth: the library that answers "who are you?"
// Hashing passwords, making sessions, the cookie, expiry, logging out:
// all of it happens in here, so we don't write it by hand like Tuesday.
// It does NOT answer "what are you allowed to do?" That's still our code.
import { betterAuth } from "better-auth";
import { NeonDatabaseProvider } from "../providers/neon/neonDatabaseProvider";

export const auth = betterAuth({
  // The docs make a new Pool here. We already have one, so Better Auth
  // gets ours: same database, same connection string.
  database: NeonDatabaseProvider.getInstance().getPool(),

  // Log in with an email and a password.
  emailAndPassword: { enabled: true },

  // Better Auth can do far more than this app uses. These are switched
  // off completely, even for someone who is logged in. (Step 9.)
  disabledPaths: [
    "/update-user",
    "/change-email",
    "/change-password",
    "/set-password",
    "/delete-user",
    "/delete-user/callback",
  ],
});
```

C2 · What `npx auth@latest generate` wrote

↑ back to Step 7

Yours will have a different date in its name. The inside should match.

`better-auth_migrations/2026-10-29T14-33-39.433Z.sql`

```
create table "user" ("id" text not null primary key, "name" text not null, "email" text not null
unique, "emailVerified" boolean not null, "image" text, "createdAt" timestamptz default
CURRENT_TIMESTAMP not null, "updatedAt" timestamptz default CURRENT_TIMESTAMP not null);

create table "session" ("id" text not null primary key, "expiresAt" timestamptz not null, "token" text
not null unique, "createdAt" timestamptz default CURRENT_TIMESTAMP not null, "updatedAt" timestamptz
```

```
not null, "ipAddress" text, "userAgent" text, "userId" text not null references "user" ("id") on delete
cascade);

create table "account" ("id" text not null primary key, "accountId" text not null, "providerId" text
not null, "userId" text not null references "user" ("id") on delete cascade, "accessToken" text,
"refreshToken" text, "idToken" text, "accessTokenExpiresAt" timestamptz, "refreshTokenExpiresAt"
timestamptz, "scope" text, "password" text, "createdAt" timestamptz default CURRENT_TIMESTAMP not null,
"updatedAt" timestamptz not null);

create table "verification" ("id" text not null primary key, "identifier" text not null, "value" text
not null, "expiresAt" timestamptz not null, "createdAt" timestamptz default CURRENT_TIMESTAMP not null,
"updatedAt" timestamptz default CURRENT_TIMESTAMP not null);

create index "session_userId_idx" on "session" ("userId");

create index "account_userId_idx" on "account" ("userId");

create index "verification_identifier_idx" on "verification" ("identifier");
```

C3 · The route, straight from the docs

↑ back to Step 8

app/api/auth/[...all]/route.ts

```
// /api/auth/... -- the addresses Better Auth answers.
// [...all] catches anything under /api/auth: /api/auth/sign-in/email,
// /api/auth/sign-up/email, /api/auth/sign-out, /api/auth/get-session...
import { auth } from "@/lib/auth";
import { toNextJsHandler } from "better-auth/next-js";

export const { POST, GET } = toNextJsHandler(auth);
```

C4 · The route, deny by default

↑ back to Step 9

app/api/auth/[...all]/route.ts

```
// /api/auth/... -- the addresses Better Auth answers.
// [...all] catches anything under /api/auth: /api/auth/sign-in/email,
// /api/auth/sign-up/email, /api/auth/sign-out, /api/auth/get-session...
import { toNextJsHandler } from "better-auth/next-js";
import { auth } from "@/lib/auth";

// The docs stop at this one line, which opens EVERY address Better Auth
// knows -- about thirty -- when our pages call four of them.
const betterAuth = toNextJsHandler(auth);

// Deny by default (Tuesday's rule): only the addresses we actually use.
// Anything else gets a 404 and never reaches Better Auth at all.
const allowed = {
  GET: ["/get-session"],
```

```
  POST: ["/sign-up/email", "/sign-in/email", "/sign-out"],
};

// "/api/auth/sign-in/email" -> "/sign-in/email"
function pathOf(request: Request) {
  return new URL(request.url).pathname.replace("/api/auth", "");
}

export async function GET(request: Request) {
  if (!allowed.GET.includes(pathOf(request))) {
    return new Response("Not found", { status: 404 });
  }
  return betterAuth.GET(request);
}

export async function POST(request: Request) {
  if (!allowed.POST.includes(pathOf(request))) {
    return new Response("Not found", { status: 404 });
  }
  return betterAuth.POST(request);
}
```

C5 · The browser half

↑ back to Step 10

lib/auth-client.ts

```
// The browser half of Better Auth. Pages call authClient.signIn.email(...)
// and it sends the request to /api/auth/... for them.
import { createAuthClient } from "better-auth/react";

export const authClient = createAuthClient();
```

C6 · The login screen

↑ back to Step 11 · C14 is the Part 4 version

app/login/page.tsx

```
"use client";

import { useState } from "react";
import { authClient } from "../../lib/auth-client";

// The login screen. authClient sends each button's request to Better Auth
// at /api/auth/..., and Better Auth sets (or clears) the cookie.
export default function Login() {
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [message, setMessage] = useState("");

  async function signUp() {
```

```
    const { data, error } = await authClient.signUp.email({ name, email, password });
    if (error) {
      setMessage(error.message ?? "Could not sign up");
      return;
    }
    // Signing up logs you straight in, so there's already a cookie.
    setMessage("Signed up and logged in as " + data.user.name);
  }

  async function logIn(e: React.FormEvent) {
    e.preventDefault();
    const { data, error } = await authClient.signIn.email({ email, password });
    if (error) {
      setMessage(error.message ?? "Could not log in");
      return;
    }
    setMessage("Logged in as " + data.user.name);
  }

  async function logOut() {
    await authClient.signOut();
    setMessage("Logged out");
  }

  return (
    <main>
      <p className="status">{message}</p>
      <form onSubmit={logIn} className="card login">
        <label>
          Name <span className="hint">(only needed to sign up)</span>
          <input value={name} onChange={(e) => setName(e.target.value)} />
        </label>
        <label>
          Email
          <input value={email} onChange={(e) => setEmail(e.target.value)} />
        </label>
        <label>
          Password
          <input type="password" value={password} onChange={(e) => setPassword(e.target.value)} />
        </label>
        <div className="actions">
          <button type="submit">Log in</button>
          <button type="button" onClick={signUp}>
            Sign up
          </button>
        </div>
      </form>
      <p>
        <button onClick={logOut}>Log out</button>
      </p>
    </main>
  );
}
```

C7 · Who is this?

↑ back to Step 12

services/authService.ts

```
// The ONE file that knows who is logged in. Same job as Tuesday's.
// Routes ask this file "who is this?" and never read the cookie themselves.
import { headers } from "next/headers";
import { auth } from "../lib/auth";

export class AuthService {
  private static instance: AuthService;

  public static getInstance(): AuthService {
    if (!AuthService.instance) {
      AuthService.instance = new AuthService();
    }
    return AuthService.instance;
  }

  // Who sent this request? Better Auth reads the cookie, looks the session
  // up in the session table, and checks it hasn't expired.
  // Gives back undefined when nobody is logged in -- or when the cookie
  // is fake, expired, or for a session that was logged out.
  public async getCurrentUser() {
    const result = await auth.api.getSession({ headers: await headers() });
    if (!result) {
      return undefined;
    }
    return {
      id: result.user.id,
      name: result.user.name,
      email: result.user.email,
    };
  }
}
```

C8 · Can you get in?

↑ back to Step 12

app/api/my-pets/route.ts

```
// GET /api/my-pets -- every pet the logged-in person has adopted.
// "My" pets only means something once the server knows who "me" is.
import { AuthService } from "../../../services/authService";
import { PetService } from "../../../services/petService";

export async function GET() {
  // Can you get in? Better Auth checks the cookie against the session table.
  const user = await AuthService.getInstance().getCurrentUser();
  if (!user) {
    return Response.json({ error: "Log in first" }, { status: 401 });
  }
```

```
  // "Mine" comes from the session, never from the request. There's no
  // ?email= here for anyone to change.
  const pets = await PetService.getInstance().getPetsOwnedBy(user.email);
  return Response.json(pets);
}
```

C9 · Is this yours?

↑ back to Step 13

app/api/my-pets/[id]/route.ts

```
// GET /api/my-pets/4 -- one pet's full record: microchip, vet notes, owner.
// Private information, so two questions, in this order.
import { AuthService } from "../../../../services/authService";
import { PetService } from "../../../../services/petService";

export async function GET(
  request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  // 1. Can you get in? Nobody logged in: 401.
  const user = await AuthService.getInstance().getCurrentUser();
  if (!user) {
    return Response.json({ error: "Log in first" }, { status: 401 });
  }

  const { id } = await params;
  const pet = await PetService.getInstance().getPetRecord(Number(id));

  // 2. Do you own this record? Logged in isn't enough. It has to be YOURS.
  // Someone else's pet gets the same 404 as a pet that doesn't exist,
  // so we don't even admit it's there.
  if (!pet || pet.ownerEmail !== user.email) {
    return Response.json({ error: `No pet of yours with id ${id}` }, { status: 404 });
  }
  return Response.json(pet);
}
```

C10 · The cookie check (Part 4 deletes it again)

↑ back to Step 15

proxy.ts

```
// proxy.ts -- runs before every request to the addresses in config.matcher.
// No login cookie? Send them to the login page instead of /my-pets.
import { NextResponse } from "next/server";
import type { NextRequest } from "next/server";
import { getSessionCookie } from "better-auth/cookies";

export function proxy(request: NextRequest) {
  // This only checks that a cookie with the right NAME is there.
```

```
  // It never asks the database whether the session is real.
  if (!getSessionCookie(request)) {
    return NextResponse.redirect(new URL("/login", request.url));
  }
  return NextResponse.next();
}

export const config = {
  matcher: ["/my-pets"],
};
```

C11 · The wrapper

↑ back to Step 17

app/UserContext.tsx

```
"use client";

// UserContext: who is logged in, handed to every page and component
// without passing it down as a prop through each one.
// The layout fills it in on the server, from Better Auth's real check.
import { createContext } from "react";

export type User = {
  id: string;
  name: string;
  email: string;
};

// null means nobody is logged in.
export const UserContext = createContext<User | null>(null);

// The wrapper. The layout is a server component, and server components
// can't hold a context, so this small client component does it for them.
export function UserProvider({
  user,
  children,
}: {
  user: User | null;
  children: React.ReactNode;
}) {
  return <UserContext value={user}>{children}</UserContext>;
}
```

C12 · The layout fills it in

↑ back to Step 18

app/layout.tsx

```
import type { Metadata } from "next";
import Link from "next/link";
import { AuthService } from "../services/authService";
```

```
import { UserProvider } from "./UserContext";
import "./globals.css";

export const metadata: Metadata = {
  title: "Paws & Whiskers",
  description: "The shelter's pets, and a private page for the people who adopted one",
};

// async now: before drawing any page, ask Better Auth who this is.
// That's the real check -- cookie, session table, expiry -- once per page.
export default async function RootLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  const user = await AuthService.getInstance().getCurrentUser();

  return (
    <html lang="en">
      <body>
        <UserProvider user={user ?? null}>
          <header>
            <h1>
              <Link href="/">Paws &amp;amp; Whiskers</Link>
            </h1>
            <nav>
              <Link href="/">At the shelter</Link> &middot; <Link href="/my-pets">My pets</Link>{" "}
              &middot; <Link href="/login">{user ? "Hi, " + user.name : "Log in"}</Link>
            </nav>
          </header>
          {children}
        </UserProvider>
      </body>
    </html>
  );
}
```

C13 · My pets, from the context

↑ back to Step 19

app/my-pets/page.tsx

```
"use client";

import Link from "next/link";
import { useContext, useEffect, useState } from "react";
import type { PetRecord } from "../../types/pet";
import { UserContext } from "../UserContext";

// The page that needs a login: the pets YOU adopted, with their private
// records. It asks UserContext who you are, and the layout filled that in
// from Better Auth's real check -- not from "is there a cookie?"
export default function MyPets() {
```

```
const user = useContext(UserContext);
const [pets, setPets] = useState<PetRecord[]>([]);
const [error, setError] = useState("");

useEffect(() => {
  async function load() {
    const res = await fetch("/api/my-pets");
    if (!res.ok) {
      // Show whatever the API said, so you can see which check stopped you.
      const body = await res.json();
      setError(res.status + ": " + body.error);
      return;
    }
    setPets(await res.json());
  }
  // Nobody logged in? Don't even ask the API.
  if (user) {
    load();
  }
}, [user]);

if (!user) {
  return (
    <main>
      <p>
        This page is only for people who adopted a pet. <Link href="/login">Log in</Link> to see
        yours.
      </p>
    </main>
  );
}

return (
  <main>
    <p className="tagline">{user.name}&apos;s pets, and their private records.</p>
    {error && <p className="error">{error}</p>}
    {pets.map((pet) => (
      <div key={pet.id} className="card">
        <h2>
          {pet.name} <span className="species">{pet.species}, {pet.age}</span>
        </h2>
        <p className="about">{pet.about}</p>
        <dl>
          <dt>Adopted</dt>
          <dd>{pet.adoptedOn}</dd>
          <dt>Microchip</dt>
          <dd>{pet.microchip}</dd>
          <dt>Vet notes</dt>
          <dd>{pet.vetNotes}</dd>
        </dl>
        <a href={"/api/my-pets/" + pet.id}>Raw record from the API</a>
      </div>
    ))}
  </main>
```

```
  );
}
```

C14 · The login screen, from the context

↑ back to Step 19

app/login/page.tsx

```
"use client";

import { useRouter } from "next/navigation";
import { useContext, useState } from "react";
import { authClient } from "../../lib/auth-client";
import { UserContext } from "../UserContext";

// The login screen. authClient sends each button's request to Better Auth
// at /api/auth/..., and Better Auth sets (or clears) the cookie.
// Who is logged in now comes from UserContext, not from a message we set.
export default function Login() {
  const user = useContext(UserContext);
  const router = useRouter();
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [message, setMessage] = useState("");

  async function signUp() {
    const { error } = await authClient.signUp.email({ name, email, password });
    if (error) {
      setMessage(error.message ?? "Could not sign up");
      return;
    }
    setMessage("");
    // The layout read the user before this login happened. refresh()
    // runs it again, so UserContext (and the header) catch up.
    router.refresh();
  }

  async function logIn(e: React.FormEvent) {
    e.preventDefault();
    const { error } = await authClient.signIn.email({ email, password });
    if (error) {
      setMessage(error.message ?? "Could not log in");
      return;
    }
    setMessage("");
    router.refresh();
  }

  async function logOut() {
    await authClient.signOut();
    router.refresh();
  }
```

```
  return (
    <main>
      <p className="status">
        {user ? "Logged in as " + user.name + " (" + user.email + ")" : "Not logged in"}
      </p>
      {message && <p className="error">{message}</p>}
      <form onSubmit={logIn} className="card login">
        <label>
          Name <span className="hint">(only needed to sign up)</span>
          <input value={name} onChange={(e) => setName(e.target.value)} />
        </label>
        <label>
          Email
          <input value={email} onChange={(e) => setEmail(e.target.value)} />
        </label>
        <label>
          Password
          <input type="password" value={password} onChange={(e) => setPassword(e.target.value)} />
        </label>
        <div className="actions">
          <button type="submit">Log in</button>
          <button type="button" onClick={signUp}>
            Sign up
          </button>
        </div>
      </form>
      <p>
        <button onClick={logOut}>Log out</button>
      </p>
    </main>
  );
}
```

C15 · Going further: vets see every pet

↑ back to Going further

services/authService.ts

```
// The ONE file that knows who is logged in. Same job as Tuesday's.
// Routes ask this file "who is this?" and never read the cookie themselves.
import { headers } from "next/headers";
import { auth } from "../lib/auth";
import { NeonDatabaseProvider } from "../providers/neon/neonDatabaseProvider";

export class AuthService {
  private static instance: AuthService;

  public static getInstance(): AuthService {
    if (!AuthService.instance) {
      AuthService.instance = new AuthService();
    }
    return AuthService.instance;
```

```
  }

  // Who sent this request? Better Auth reads the cookie, looks the session
  // up in the session table, and checks it hasn't expired.
  // Gives back undefined when nobody is logged in -- or when the cookie
  // is fake, expired, or for a session that was logged out.
  public async getCurrentUser() {
    const result = await auth.api.getSession({ headers: await headers() });
    if (!result) {
      return undefined;
    }
    return {
      id: result.user.id,
      name: result.user.name,
      email: result.user.email,
    };
  }

  // Is this person one of the shelter's vets? Better Auth has never heard
  // of vets. This is authorization, and it's ours: a row in our vets table.
  public async isVet(email: string) {
    const result = await NeonDatabaseProvider.getInstance()
      .getPool()
      .query("SELECT 1 FROM vets WHERE email = $1", [email]);
    return result.rows.length > 0;
  }
}
```

app/api/my-pets/[id]/route.ts

```
// GET /api/my-pets/4 -- one pet's full record: microchip, vet notes, owner.
// Private information, so two questions, in this order.
// Going further: a vet can see every pet's record, not just their own.
import { AuthService } from "../../../../services/authService";
import { PetService } from "../../../../services/petService";

export async function GET(
  request: Request,
  { params }: { params: Promise<{ id: string }> }
) {
  // 1. Can you get in? Nobody logged in: 401.
  const user = await AuthService.getInstance().getCurrentUser();
  if (!user) {
    return Response.json({ error: "Log in first" }, { status: 401 });
  }

  const { id } = await params;
  const pet = await PetService.getInstance().getPetRecord(Number(id));
  if (!pet) {
    return Response.json({ error: `No pet of yours with id ${id}` }, { status: 404 });
  }

  // 2. Are you allowed this record? Yours, or you're a vet.
  // Ask the cheap question first: only check the vets table if it isn't yours.
  const isOwner = pet.ownerEmail === user.email;
```

```
  if (!isOwner && !(await AuthService.getInstance().isVet(user.email))) {
    return Response.json({ error: `No pet of yours with id ${id}` }, { status: 404 });
  }
  return Response.json(pet);
}
```

MTC AI-Enabled Full-Stack Developer program · Week 14 lab.